.UBAfiuba

FACULTAD DE INGENIERÍA

Estructuras Iterativas en C

Fundamentos de Programación

rev1.1

Estructuras iterativas:

Disponemos de instrucciones con las cuales podemos lograr que un grupo de acciones se ejecuten repetidamente.

Estas instrucciones son: for, while, do..while

For:

Para lograr repetir la ejecución de un conjunto de acciones, **un determinado número de veces**.

Antes de comenzar el bloque iterativo sabemos exactamente cuántos ciclos debemos realizar.

For (sintaxis):

```
for (inicialización; condición; control) {
   // bloque de acciones a repetir
}
```

En la sentencia de **inicialización**, asignaremos el valor inicial a la variable de control.

La sentencia **condición** debe depender de la variable de control, y mientras sea verdadera se continua iterando.

En la sentencia de **control** se actualiza la variable de control, la cual se ejecuta al finalizar cada iteración.

For (Ejemplo):

```
#include <stdio.h>
int main()
{
    int i, n = 10;
    for (i=0; i<n; i++) {
        printf("%d ", i);
    }
}
```

Se conoce la cantidad exacta de veces que se realizará la iteración.

La variable i tomará los valores entre [0,9] por lo que el bloque se ejecutará 10 veces.

While:

Cuando queremos repetir una ó más acciones, **mientras una condición sea verdadera, y no sabemos en qué momento se hará falsa**.

Desconocemos cuándo sucederá el evento que hará falsa la condición.

Las acciones pueden ejecutarse múltiples veces, una vez o ningúna.

While (sintaxis):

```
while (condición) {
   // bloque de acciones a repetir
   // eventual modificación de la variable de control
}
```

La sentencia **condición** debe ser el resultado de una expresión booleana, mientras la condición sea verdadera se repetirá la ejecución del bloque de acciones.

La variable de control eventualmente debe ser modificada para que la condición sea falsa y finalice la instrucción while. En caso de nunca modificarse las variables de control se puede caer en **un ciclo infinito lo cual es un error.**

While (Ejemplo):

```c
#include <stdio.h>
int main()
{
    int num;
    printf("Ingrese número: ");
    scanf("%d", &num);
    while ((num % 2) == 0) {
        printf("Numero par: %d \n", num);
        printf("Ingrese número: ");
        scanf("%d", &num);
    }
    printf("Se ingresó un número impar.\n");
    printf("Fin del ingreso de datos.\n");
}
```

Se muestran los números ingresados mientras se cumpla la condición, en este caso si el número ingresado es par.

Cuando se ingresa un número impar **(se desconoce en qué momento puede suceder)**, la condición pasa a ser falsa y el while finaliza.

Do..While:

Cuando queremos que un bloque de acciones **se ejecute al menos una vez,** y luego pueda repetirse **mientras una condición sea verdadera, y no sabemos en qué momento se hará falsa**.

Desconocemos cuándo sucederá el evento que hará falsa la condición.

Las acciones pueden ejecutarse una o más veces.

Do..While (sintaxis):

```
do {
   // bloque de acciones a repetir
   // eventual modificación de la variable de control
} while (condición);
```

La sentencia **condición** debe ser el resultado de una expresión booleana, mientras la condición sea verdadera se repetirá la ejecución del bloque de acciones.

La variable de control eventualmente debe ser modificada para que la condición sea falsa y finalice la instrucción do..while. En caso de nunca modificarse las variables de control se puede caer en **un ciclo infinito lo cual es un error.**

Do..While (Ejemplo):

```c
#include <stdio.h>
int main()
{
    int num;
    do {
        printf("Ingrese número entre 1 y 20: ");
        scanf("%d", &num);
    } while ((num < 1) || (num > 20));
    printf("El numero ingresado es: %d\n", num);
}
```

Se solicita un número al menos una vez, y se continúa solicitando mientras el número esté fuera del rango solicitado.

Cuando se ingresa un número dentro del rango **(se desconoce en qué momento puede suceder)**, la condición pasa a ser falsa y el do..while finaliza.